Álvaro Puebla Ruisánchez / Óscar López García

CYBERSECURITY REPORT – LABORATORY 3

The 2 types of cyphers in Cryptool desktop:

- **ECB** - In this mode, the **message is split into blocks**, and the blocks are sequentially encrypted. This mode is **vulnerable to attack using frequency analysis**, the same sort used in simple substitution. Identical blocks would get encrypted to the same blocks, thus exposing the key.
- **CBC** - A logical operation is performed on the first block with what is known as an initial vector using the secret key to randomize the first block. The **output of this step is logically combined with the second block** and the key to generating encrypted text, **which is then used with the third block and so on**.

1 Block ciphers:

1.1 Compare the entropy value for each type of plaintext with ciphertext.

This are the texts we used:

A. Homogenous

hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh

B. Medium diversified

agree all use city with model agree all use city with model

C. Highly diversified

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Phasellus faucibus scelerisque eleifend donec pretium vulputate sapien nec sagittis aliquam. Quisque id diam vel quam elementum pulvinar etiam non quam lacus suspendisse. Nunc lobortis mattis aliquam faucibus purus in massa tempor nec feugiat. Integer feugiat scelerisque varius morbi enim nunc faucibus a pellentesque sit. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vivamus at augue eget arcu dictum varius duis at consectetur lorem.

Curabitur non nulla sit amet nisl tempus convallis quis ac lectus. Sed porttitor lectus nibh. Vivamus magna justo, lacinia eget consectetur sed, convallis at tellus. Praesent sapien massa, convallis a pellentesque nec, egestas non nisi. Nulla porttitor accumsan tincidunt. Pellentesque in ipsum id orci porta dapibus. Mauris blandit aliquet elit, eget tincidunt nibh pulvinar a. Donec rutrum congue leo eget malesuada. Integer ac neque id augue tincidunt suscipit.

The key that is capped to X bits: 66 91 B2 7A EA 26 3E 58 82 FC DB 56

BA B9 F5 C6 AE D0 12 D5 F0 AF AE 77

5C 66 43 25 3A 25 3A 65 C7 4C 62 BF

8F 7A F7 28 8A F4 96 63 6E 0A F3 F8

D8 5C A2 B6 04 51 D2 E8 F4 4A 79 F6

C0 D0 65 EF 66 91 B2 7A EA 26 3E 58

82 FC DB 56 BA B9 F5 C6 AE D0 12 D5

F0 AF AE 77 5C 66 43 25 3A 25 3A 65

C7 4C 62 BF 8F 7A F7 28 8A F4 96 63

6E 0A F3 F8 D8 5C A2 B6 04 51 D2 E8

F4 4A 79 F6 C0 D0 65 EF 66 91 B2 7A

EA 26 3E 58 82 FC DB 56 BA B9 F5 C6

AE D0 12 D5 F0 AF AE 77 5C 66 43 25

3A 25 3A 65 C7 4C 62 BF 8F 7A F7 28

8A F4 96 63 6E 0A F3 F8 D8 5C A2 B6

04 51 D2 E8 F4 4A 79 F6 C0 D0 65 EF

66 91 B2 7A EA 26 3E 58 82 FC DB 56

BA B9 F5 C6 AE D0 12 D5 F0 AF AE 77

5C 66 43 25 3A 25 3A 65 C7 4C 62 BF

Entropy depending in the cypher:

Text	IDEA	DES (ECB)	DES (CBC)	TripleDES (ECB)	TripleDES (CBC)	AES (CBC)	Serpent (128)	RC6 (128)	MARS (128)	Twofish (128)	**ORIGINAL**
Homogenous	3.05	3.06	7.79	3.06	7.78	7.79	7.81	7.79	7.82	7.82	0.00/8.00
Medium diversified	6.54	6.45	7.81	6.50	7.82	7.83	7.85	7.82	7.82	7.80	3.80/4.70
Highly diversified	7.82	7.86	7.87	7.85	7.85	7.85	7.87	7.85	7.87	7.86	4.00/4.70

ALGORITHM	KEY LENGTH (bits)
IDEA	128
DES	64 (56 effective)
Triple DES	128 (112 effective)
AES	128 – 192 – 256
SERPENT	128 – 192 – 256
RC6	128 – 192 – 256
MARS	128 – 192 – 256
Twofish	128 – 192 – 256

1.2 Compare the impact of a block and key length on the entropy of ciphertext. If applicable, try to use different available lengths and key values of selected algorithms.

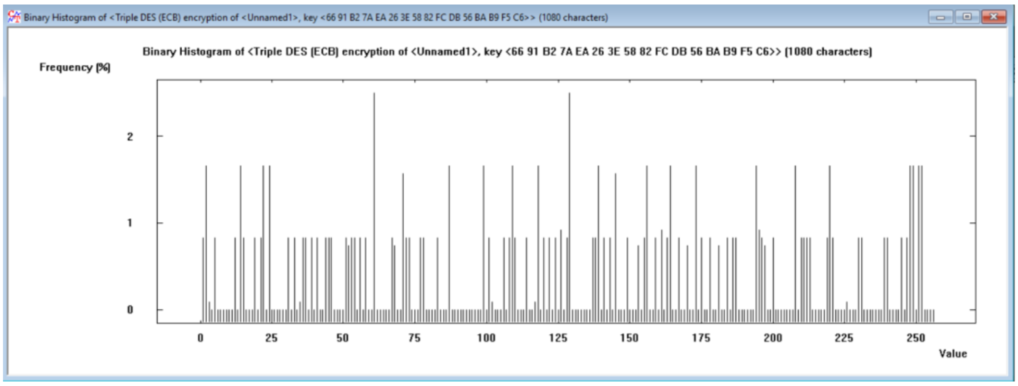
- **Block length**: **small block sizes** (e.g. DES 64-bit) + **deterministic mode (ECB)** increase repeated ciphertext blocks for repetitive plaintext -> lower observed entropy and visible patterns. Larger block sizes and randomized/chaining modes hide patterns and push entropy toward max.
- **Key length**: for a correct, secure cipher, changing key length does **not** materially change per-byte ciphertext entropy; **it changes brute-force resistance**, not statistical randomness of the ciphertext.

1.3 Which algorithms are most popular? Which parameter values (block length, key length) are nowadays considered standard (safe)?

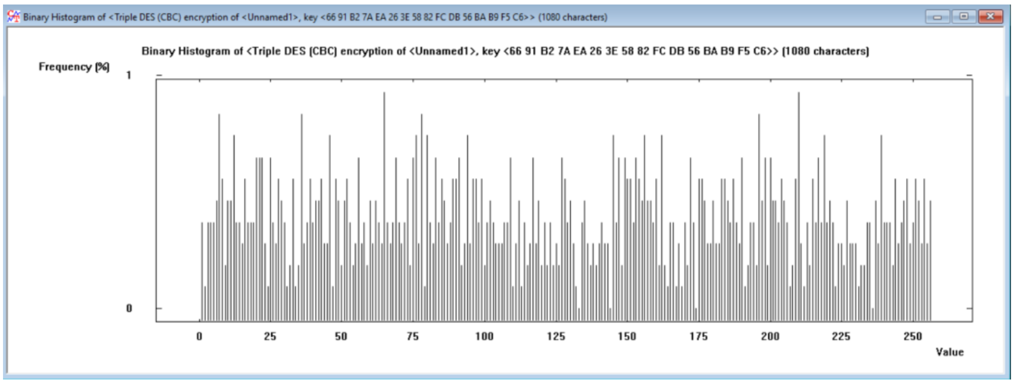
- Widely used / standard: **AES** (mandatory), ChaCha20 (stream, in TLS/SSH/etc.), AES-GCM for authenticated encryption.
- Safe parameters today: **AES with 128 or 256-bit keys**; **AES block size = 128 bits**. **Avoid DES** (56-bit key) and 3DES for new designs (deprecated). Use authenticated modes (GCM/CCM/AEAD) **rather than raw CBC/ECB**.

1.4 What can we say about the observed changes in histograms and entropy values from tasks 1 and 2?

- Homogenous plaintext + ECB → histogram with spikes, lower entropy.



- Same plaintext + CBC/CFB/OFB/CTR → flattened histogram, entropy ≈ 8.0 (per byte).



Conclusion: mode and block size determine observable statistical uniformity; good modes mask plaintext structure so entropy metrics approach maximum.

1.5 What can we say about the observed changes in histograms and entropy values during the realization similar task for historical algorithms?

Historical ciphers (DES, 3DES) can produce high per-byte entropy for diversified plaintext, but:

- Small block size (DES 64-bit) + ECB **more likely to reveal structure**.
- Security weaknesses are about key space/attacks, not per-byte entropy; **patterns are easier to exploit with small blocks**/misuse.

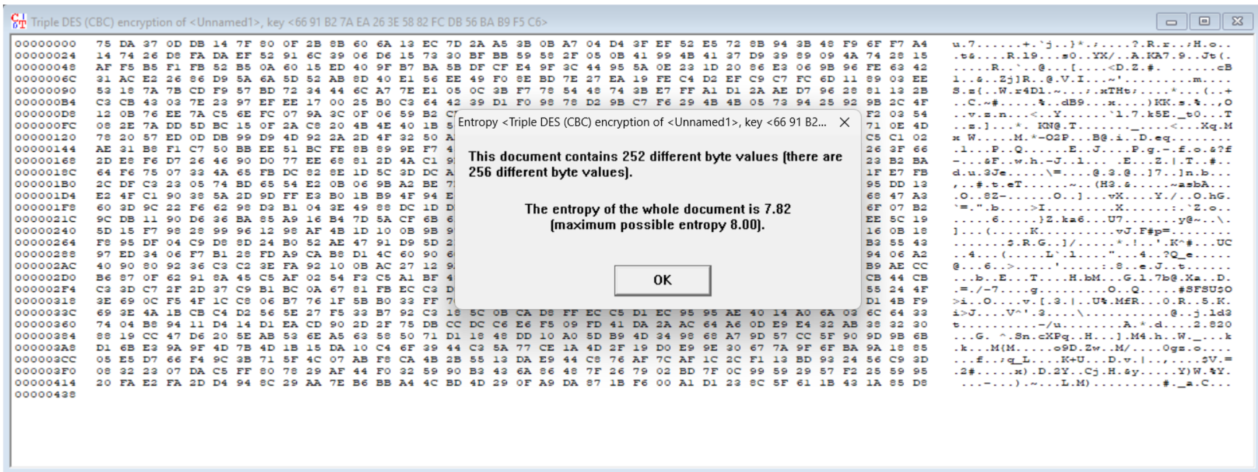
1.6 Does the length of the block affect the entropy of the ciphertext?

Yes, indirectly. Smaller blocks make repeated plaintext more likely to map to repeated ciphertext blocks in deterministic modes (**lower observed entropy / visible patterns**). With randomized/stream modes, block length impact on observed per-byte entropy is negligible.

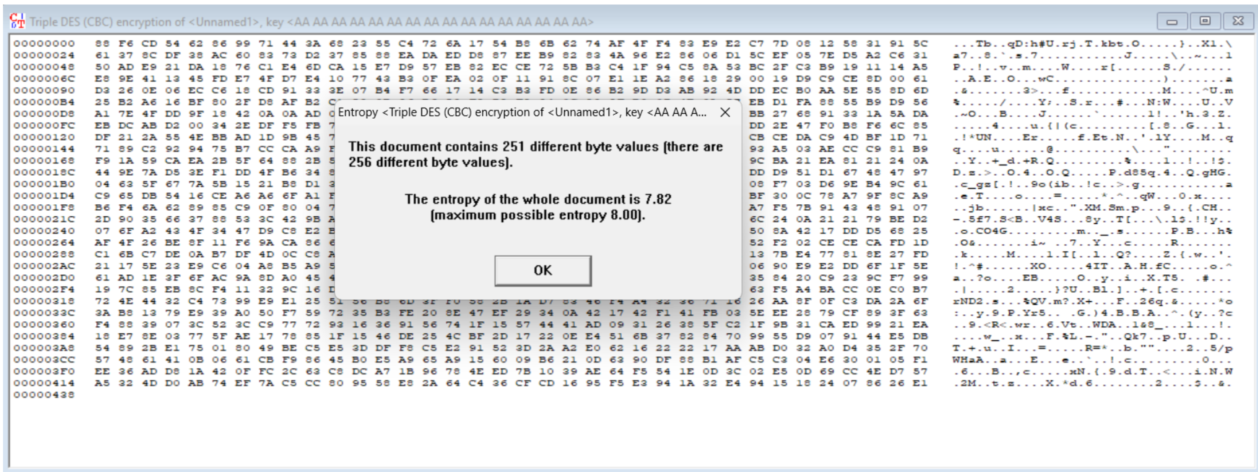
1.7 Does the length of the key influence the entropy of the ciphertext?

No meaningful effect on observed entropy for correct ciphers. All secure keys yield ciphertext with near-uniform distribution; key length affects security against brute-force only.

Example: medium text key: 6691B27AEA263E5882FCDB56BAB9F5C6



Same text with key: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA



Both have the same entropy even if one has a monoalphabetic key.

1.8 Does the observed entropy of a ciphertext depend on the entropy of plain text?

Partly. Low-entropy plaintext (e.g., all-same byte) can produce lower observed ciphertext entropy in **deterministic** modes (**ECB**), but with proper randomization (CBC) ciphertext entropy is driven to near maximum regardless of plaintext entropy.

1.9 Does the observed ciphertext entropy depend on the algorithm used?

Yes, if the algorithm or its mode is weak (ECB)/misconfigured. Proper modern ciphers produce near uniform ciphertext; weak ciphers, small blocks, deterministic modes, or poor IV (initialization vector) use can **reduce** observed **entropy** and reveal structure.

2 Modes of operation:

2.1 Generate a homogenous text (only one letter) and a medium-diversified text – a recurring sequence of characters e.g. "abcd," etc.

This are the texts we used:
A. Homogenous

hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh

B. Medium diversified

agree all use city with model agree all use city with model

C. Highly diversified

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Phasellus faucibus scelerisque eleifend donec pretium vulputate sapien nec sagittis aliquam. Quisque id diam vel quam elementum pulvinar etiam non quam lacus suspendisse. Nunc lobortis mattis aliquam faucibus purus in massa tempor nec feugiat. Integer feugiat scelerisque varius morbi enim nunc faucibus a pellentesque sit. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vivamus at augue eget arcu dictum varius duis at consectetur lorem.

Curabitur non nulla sit amet nisl tempus convallis quis ac lectus. Sed porttitor lectus nibh. Vivamus magna justo, lacinia eget consectetur sed, convallis at tellus. Praesent sapien massa, convallis a pellentesque nec, egestas non nisi. Nulla porttitor accumsan tincidunt. Pellentesque in ipsum id orci porta dapibus. Mauris blandit aliquet elit, eget tincidunt nibh pulvinar a. Donec rutrum congue leo eget malesuada. Integer ac neque id augue tincidunt suscipit.

2.2 <u>Choose one of the available encryption algorithms and encrypt a plain text (highly-diversified text) using a different mode of operations, i.e. create ciphertexts for the modes: ECB, CBC, OFB, CFB.</u>

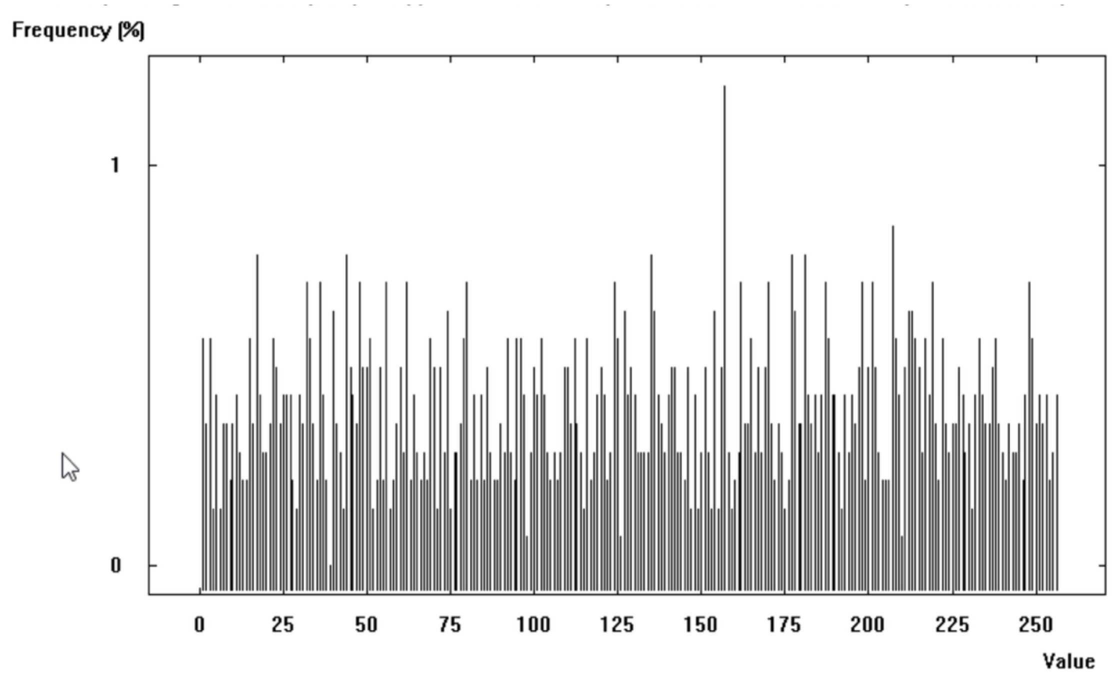
Solved bellow:

2.3 Check the histograms and calculate the entropy for the previously created ciphertexts.

Key: A2 F3 2D 87 11 AC BB 01

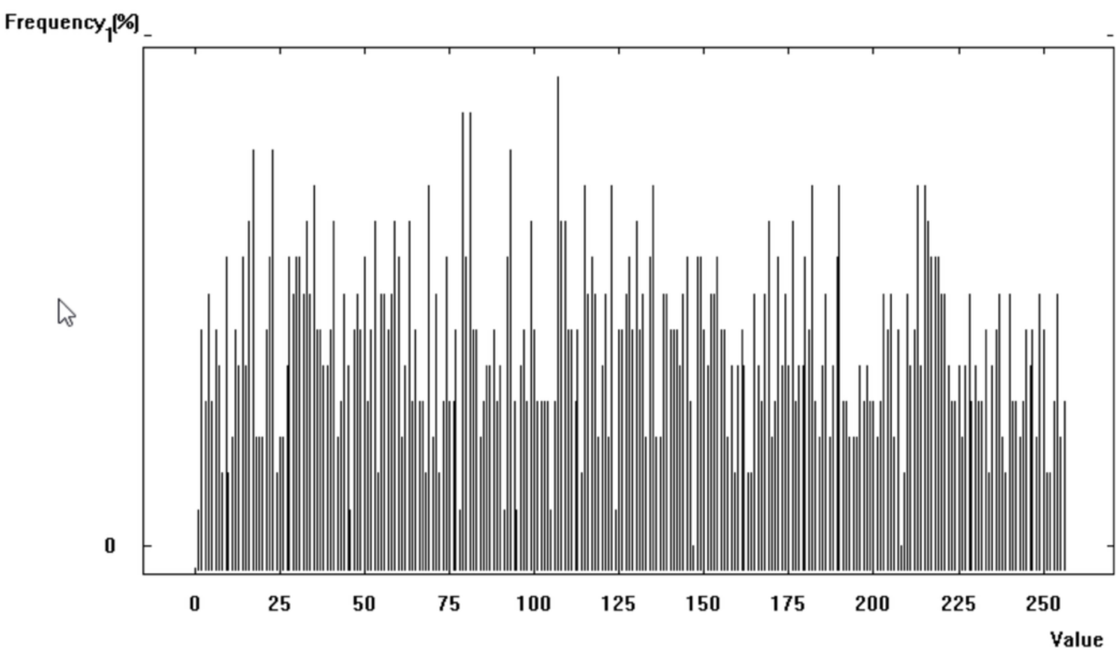
CBC:

- Entropy: 7.85
- Histogram:



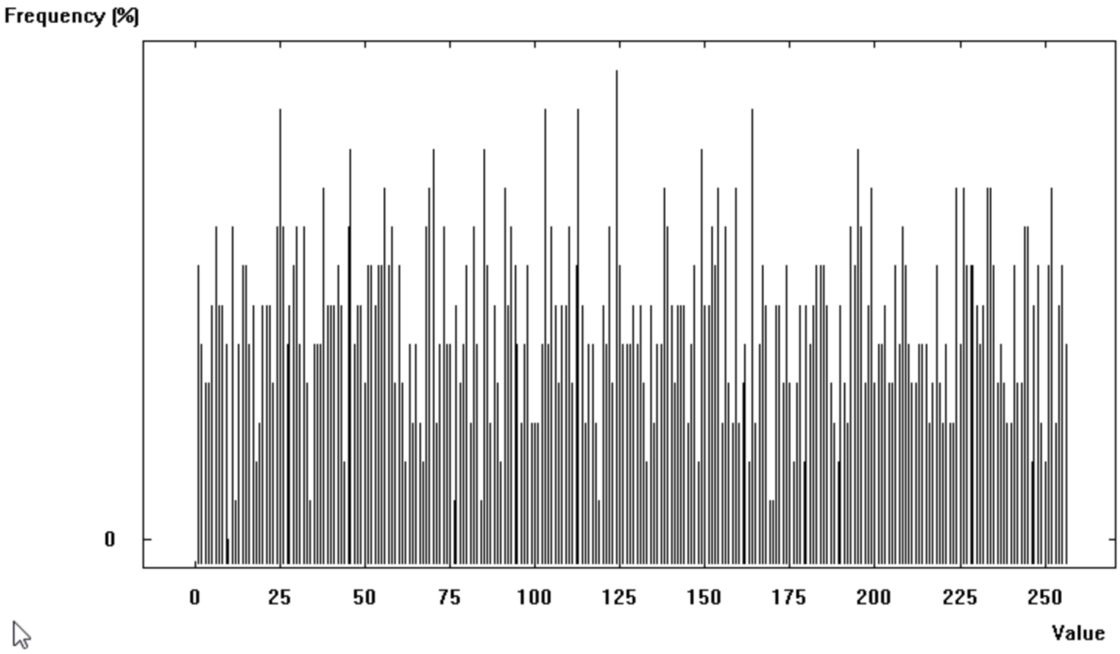
ECB:

- Entropy: 7.85
- Histogram:



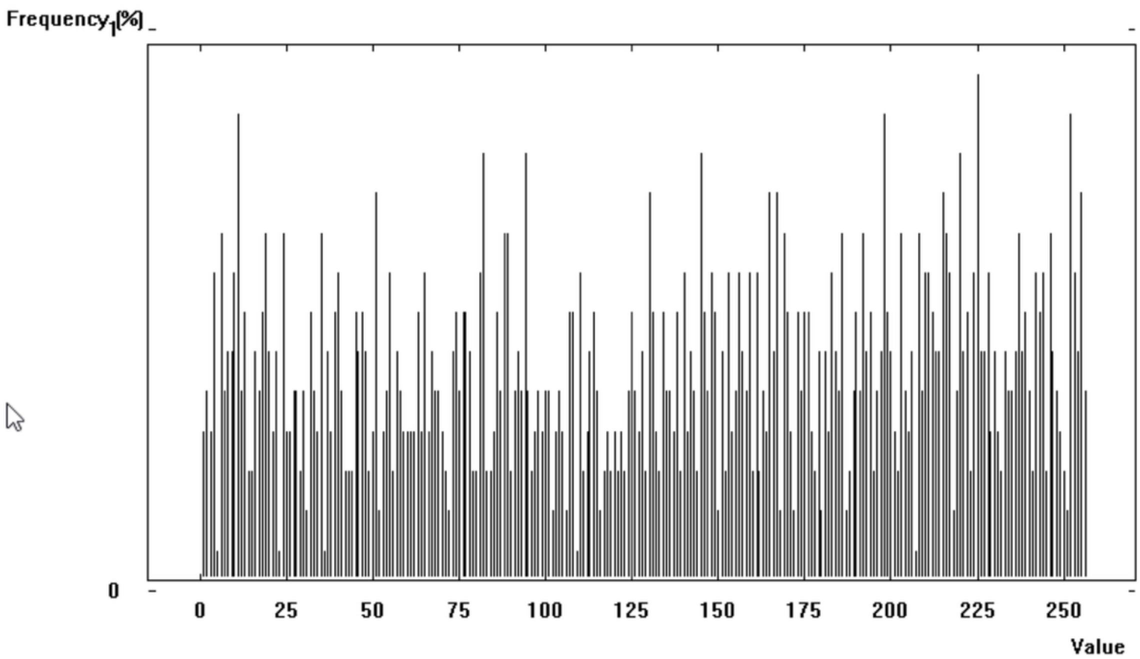
OFB:

- Entropy: 7.87
- Histogram:



CFB:

- Entropy: 7.86
- Histogram:



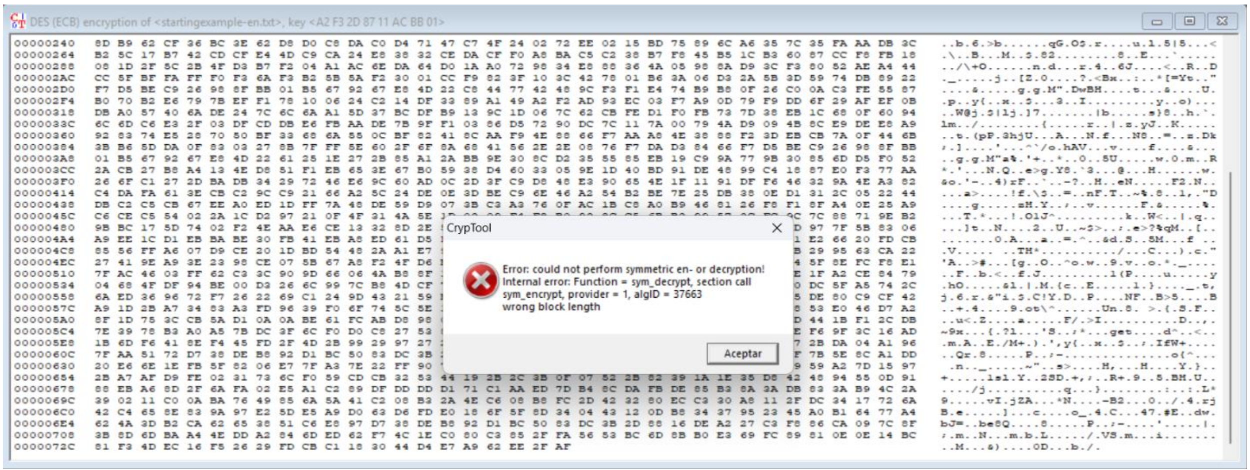
2.4 Check the contents of the ciphertext and evaluate the suitability of each mode of operation for encrypting files consisting of repeating blocks of bytes.

When using **ECB** to encript, we can **see how for repeating input blocks**, the **same output blocks**. This reveals that ECB is not suitable for encripting repenting patterns. On the other hand, the rest of algorithms remove visible repetitions, so they are more secure and reliable for encripting.

2.5 Encrypt any plain text (highly-diversified text) using AES or DES algorithm and make the following changes to the ciphertext:

- add one byte,

It shows an error as the algorithm knows that the length is not divisible by the block size.

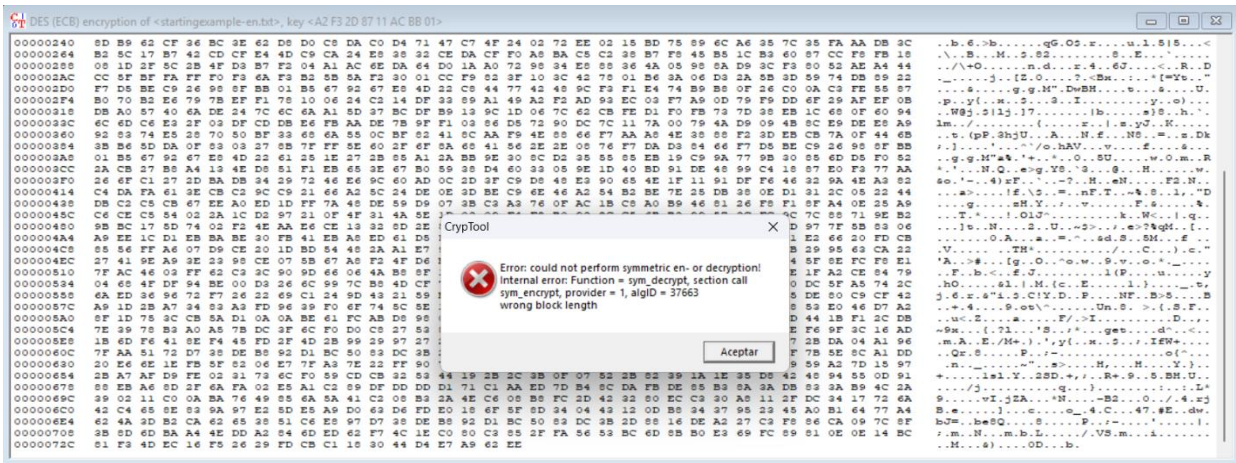


- remove one byte,

It shows another error as the algorithm knows that the length is not divisible by the block size.

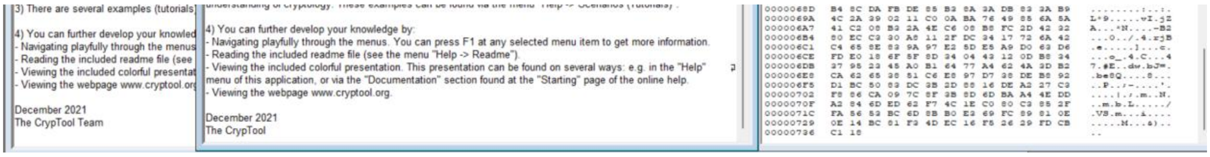
- change one each or several bits in different bytes (close or far apart),

Does not matter as far as the size of the text is not divisible by the block size.



- <u>add/remove a piece of ciphertext equal to the length of the algorithm block,</u>

After **removing the block size in length it works** decrypting only what it remains. Eliminating " Team" from the plain text.



<u>Decrypt the ciphertexts and check their content.</u>

2.6 <u>Please try for the following cryptograms to identify and justify your decision on the:</u>

- <u>the mode of operation of the algorithm (ECB,CBC,OFB,CFB),</u>
- <u>the block length of the algorithm,</u>

a.

<u>685c281585d8c0eeb59022def8488a78</u>
<u>685c281585d8c0eeb59022def8488a78</u>
<u>685c281585d8c0eeb59022def8488a78</u>
<u>685c281585d8c0eeb59022def8488a78</u>
<u>685c281585d8c0eeb59022def8488a78</u>
<u>685c281585d8c0eeb59022def8488a78</u>
<u>685c281585d8c0eeb59022def8488a78</u>

ECB, as we can see the repetitions in the cyphertext, and ECB is the only one in which the repetitions are clearly shown in the cyphertext. Length of 128 bits as every repeating pattern has **32 hexadecimal characters**. **Then, its 128 bits (ECB).**

b.

<u>43d9f1d44ebea3e411ad70dd41b2c24e</u>
<u>af20e9491c13f9d419862d7f1bceca14</u>
<u>43d9f1d44ebea3e411ad70dd41b2c24e</u>
<u>af20e9491c13f9d419862d7f1bceca14</u>
<u>43d9f1d44ebea3e411ad70dd41b2c24e</u>

af20e9491c13f9d419862d7f1bceca14
43d9f1d44ebea3e411ad70dd41b2c24e
af20e9491c13f9d419862d7f1bceca14
43d9f1d44ebea3e411ad70dd41b2c24e
af20e9491c13f9d419862d7f1bceca14

ECB, as we can see the repetitions in the cyphertext, and ECB is the only one in which the repetitions are clearly shown in the cyphertext. Length of 256 bits as every repeating pattern has **64 hexadecimal characters**. **Then, its 256 bits (ECB).**

c.

6a239123a19647032e3e637ab0b02d56
26b593edf8445c07e9e79e408ec89e56
16d9e89bef28c92d61ab6d82d39921ac
8109e9d03dafd2fd6416a0ba97513ede
0615a2688a785319d75d95a4a9aed55e
e6c221cfa51d7e89f93a5dfbc4a2bc22
368ecba0bd42c8f56c2710723b883e5

As there are no repetitions, the text if encrypted with Cryptool desktop, it must have been encrypted with **CBC** as this mode avoids repeated patterns, and the length cannot be known.

2.7 What do a ciphertext and its entropy look like for plain text with the homogeneous structure depending on the chosen encryption mode?

Given the mode **ECB**, we would **see the repetition in all the texts** as they are mantained in the cyphertext and with an entropy a little higher than the original text.

Given the mode **CBC**, we **won't see any patterns** making the **entropy nearly perfect**.

2.8 What is the impact of errors introduced into the ciphertext after decryption, what does the plain text with changed one bit and multiple bits look like?

In **ECB, only the affected block is corrupted**. In **CBC, the modified block and the next ones are damaged**. **CFB causes errors in a few bytes before recovery**, while **OFB affects only the corresponding bit**.

Thus, **ECB and OFB localize errors**, while **CBC and CFB propagate them** due to their chaining structure.

2.9 What does each mode of operation do with the loss of a part of the ciphertext? Is it possible to retrieve plain text after removing some parts from the ciphertext (add screens)?

In **ECB, each block is independent**, so any missing block only affects that portion of the plaintext; **the rest can still be decrypted correctly**. In **CBC, CFB, and OFB, decryption becomes desynchronized after the missing part** because each **block depends on previous ciphertext data**. As a result, a loss of ciphertext causes corruption or total loss of the following plaintext.

Therefore, ECB allows partial recovery, while CBC, CFB, and OFB cannot recover correctly after ciphertext loss due to their chaining or feedback mechanisms.

2.10 What is the most suitable application the following mode of operation: CBC, EBC?

We would use **ECB for random data or short and non-important data** as it is simple and fuet but insecure. If we need to encript **text files or someting important, it is much better to use CBC**.

2.11 Which of the modes of operation allows for encryption and decryption processes in parallel? Which mode of operation allows us to divide plaintext/ ciphertext into several parts to perform encryption or decryption independently (on different threads)

In **ECB** each block is processed independently, so encryption and decription can be performed in parallel. So this is the only one that we can divide into different threads in Cryptool desktop.